

Ministry of Education and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Software Engineering Department

Embedded Systems
Laboratory Work #7

**Actuators – Control, signal conditioning, hardware
inputs, event history, and latency statistics**

Author:
Alexandru Cebotari
Group FAF-233

Verified by:
Alexei Martiniuc

Technical scope

The final implementation targets Arduino Uno in PlatformIO and Wokwi and now covers the full requested software scope for the updated project state: the base condition, the supplementary requirements, and the complete two-actuator implementation.

The system includes:

- one binary actuator: LED on pin D13;
- one analog actuator: PWM LED on pin D10;
- one hardware pushbutton on pin D2 for binary input;
- one potentiometer on pin A0 for analog input;
- one LCD1602 with I2C interface;
- one serial STDIO command interface.

The application supports serial input mode, hardware input mode, and hybrid mode. In addition to actuator control, it keeps an event-history buffer and latency statistics for both actuation paths.

Objectives

- implement a binary actuator path with validation and temporal debouncing;
- implement an analog actuator path with saturation, filtering, and ramp limiting;
- control both a binary actuator and an analog actuator in the same application;
- provide local display and structured serial reporting;
- support both virtual hardware inputs and terminal commands;
- add engineering-grade observability through event history and latency statistics;
- preserve modular design and fit safely in Arduino Uno memory.

1 1. Domain analysis

1.1 1.1 Engineering context

Actuation systems rarely receive perfect commands. Binary requests can chatter and analog requests can contain spikes, overshoot, or abrupt transitions. For this reason, the project treats the software as a signal-conditioning chain, not as a direct I/O mirror.

Binary actuation is represented by a two-state LED output, which stands for a relay or another ON/OFF load. Analog actuation is represented by a PWM-controlled LED, which stands for a continuously controlled actuator such as a dimmable load, a

speed-controlled motor, or another duty-cycle-driven element. This choice keeps the implementation Wokwi-friendly while preserving the semantics required by the complete two-actuator implementation.

1.2 1.2 Why the selected hardware is valid

The binary actuator is a direct digital output and the analog actuator is a PWM output. The pushbutton and potentiometer provide realistic physical input signals. This is stronger than a serial-only demo because the project now demonstrates both command interfaces and signal conditioning for real hardware-style inputs.

1.3 1.3 Condition and supplementary-requirements coverage

The final solution addresses the three requirement groups as follows:

- **Condition:** command acquisition, actuator control, LCD display, STDIO reporting, and sub-100 ms binary response.
- **Supplementary requirements:** modular architecture, explicit conditioning stages, diagnostics, and complete documentation with diagrams.
- **Two-actuator implementation:** one binary actuator plus one analog actuator in the same application.

2. Requirements and design

2.1 Functional requirements matrix

ID	Requirement	Implemented by
R1	Receive commands through serial STDIO.	command parser + command task
R2	Control a binary actuator.	LED driver on D13
R3	Apply binary signal conditioning.	validation + debounce service
R4	Display system state on LCD.	reporting task + LCD mirror
R5	Print structured serial reports.	reporting task
R6	Implement the second actuator as an analog output path.	PWM LED on D10
R7	Apply analog signal conditioning.	saturation + median + weighted average + ramp
R8	Support physical control inputs.	button on D2, potentiometer on A0
R9	Keep event history and latency statistics.	runtime telemetry service
R10	Preserve modular implementation on Arduino Uno.	tasks, drivers, services, app layer

Table 1: Requirement coverage matrix

2.2 Architecture

The implementation is split into four layers:

- **Application layer:** cooperative scheduler and shared runtime state.
- **Task layer:** command handling, hardware input sampling, conditioning, actuation, and reporting.
- **Service layer:** command parsing, binary conditioning, analog conditioning, and telemetry.
- **Driver layer:** LED outputs, LCD, serial bridge, pushbutton, and potentiometer.

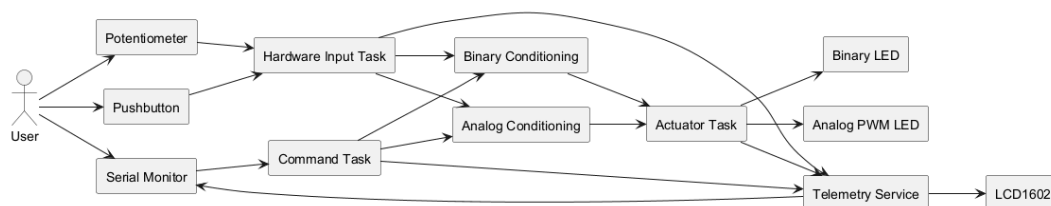


Figure 1: System structure of the final implementation

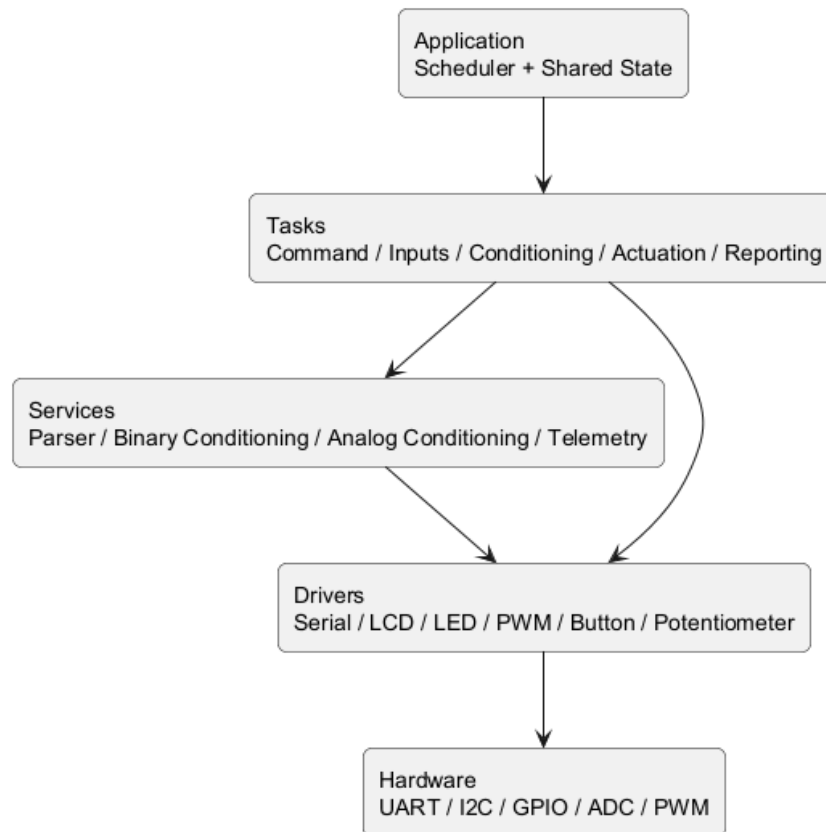


Figure 2: Layered hardware-software interface

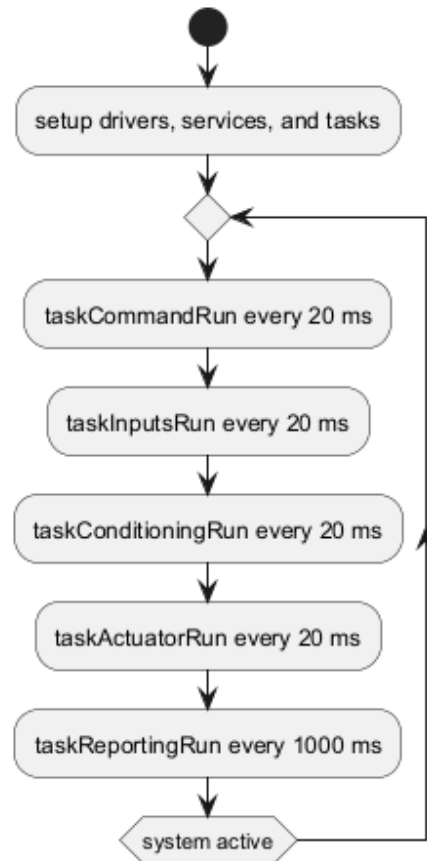


Figure 3: Cooperative scheduler with the hardware-input task

2.3 2.3 Input modes

The project supports three operating modes:

- INPUT SERIAL: only serial commands drive the actuators;
- INPUT HARDWARE: only the pushbutton and potentiometer drive the actuators;
- INPUT HYBRID: both sources are active, and the latest source update wins.

This makes the project suitable for both formal demonstration and testing.

2.4 2.4 Conditioning logic

The binary conditioning chain uses validation and temporal debouncing. The analog conditioning chain uses four stages:

1. saturation to the interval 0..100%;
2. median filtering for impulse rejection;
3. weighted averaging for smoothing;
4. ramp limiting for progressive output change.

The gradual analog change is therefore intentional and is part of the design, not an accident.

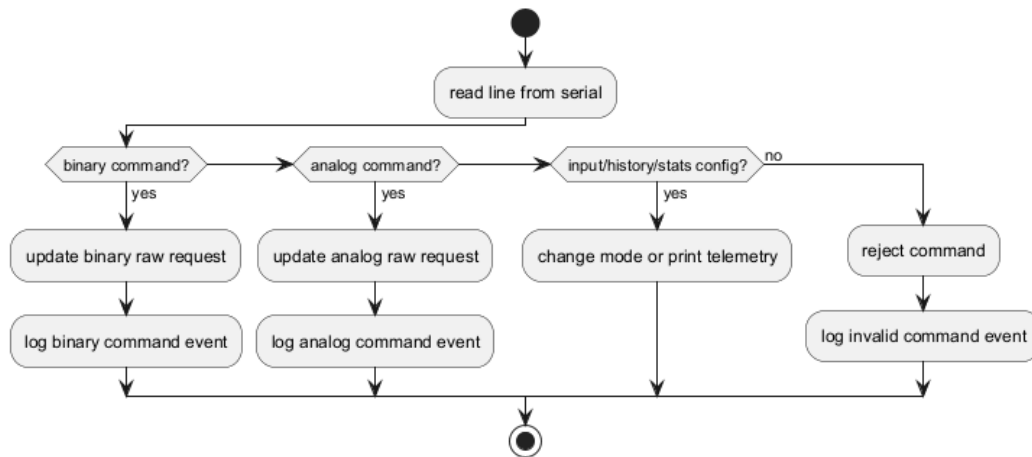


Figure 4: Command and mode-management flow

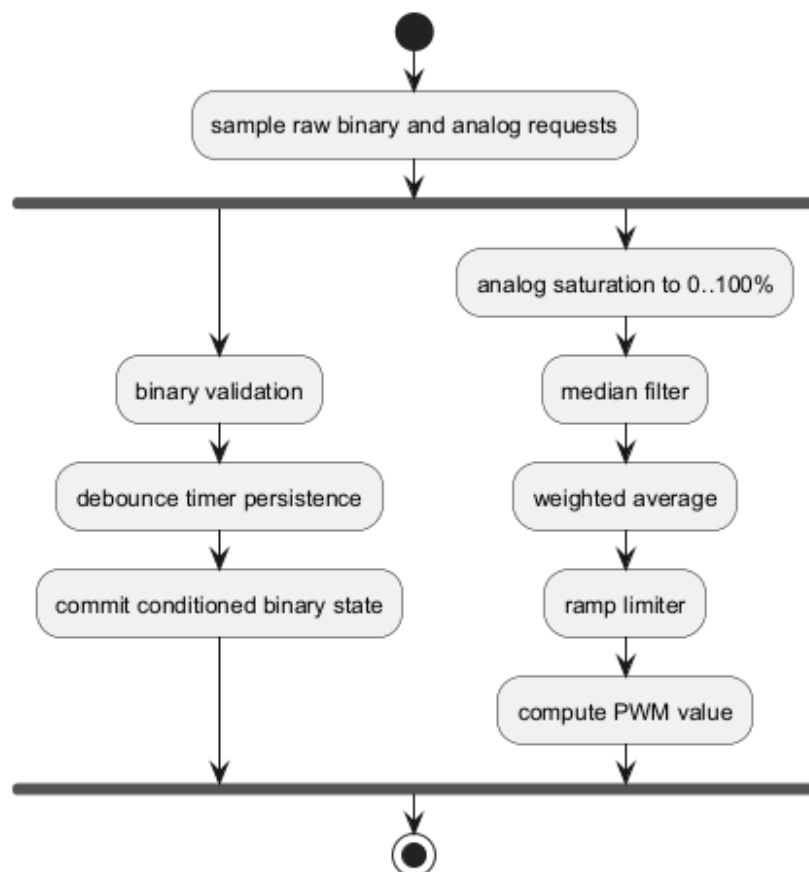


Figure 5: Binary and analog conditioning flow

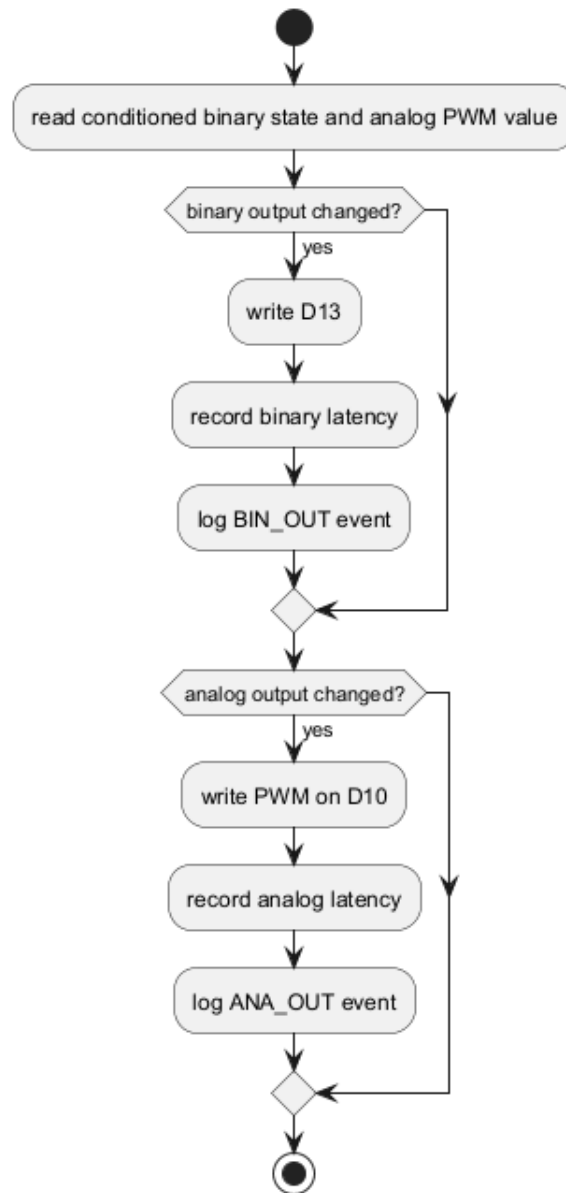


Figure 6: Actuator update flow for both outputs

2.5 2.5 Added bonus features

Two extra features were implemented on top of the base laboratory requirements:

- **Physical pushbutton + potentiometer input path:** the same actuators can be controlled from hardware, not only from the terminal.
- **Event history + latency statistics:** recent system events are stored in a ring buffer, while binary and analog command-to-output delays are accumulated as statistics.

These additions improve the technical depth of the project and create stronger validation evidence.

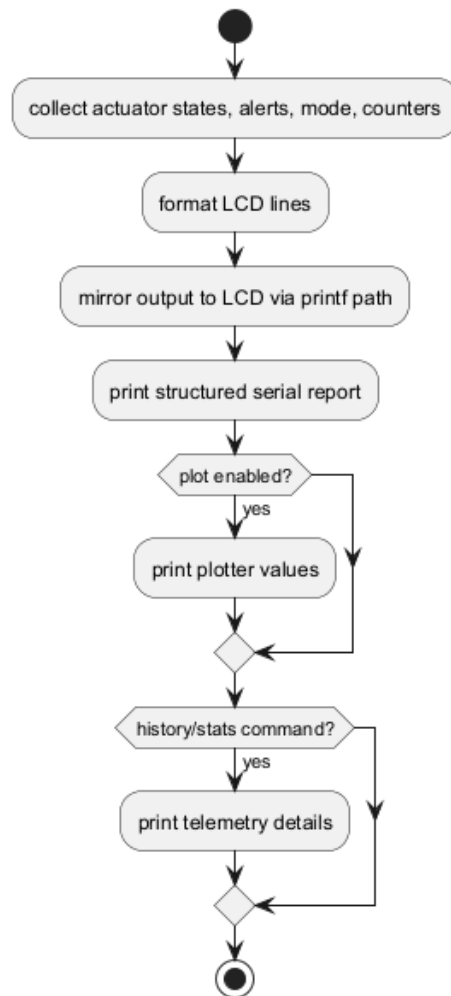


Figure 7: Reporting, event history, and statistics flow

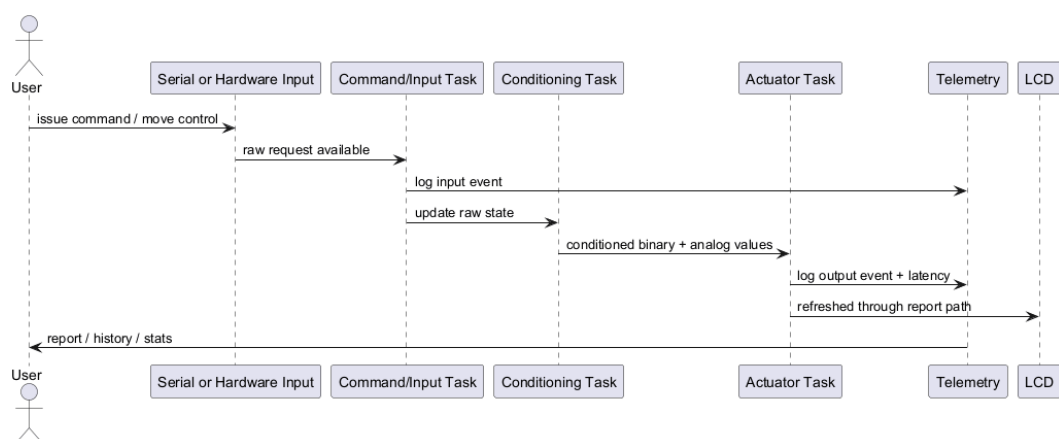


Figure 8: Typical sequence from input source to conditioned output

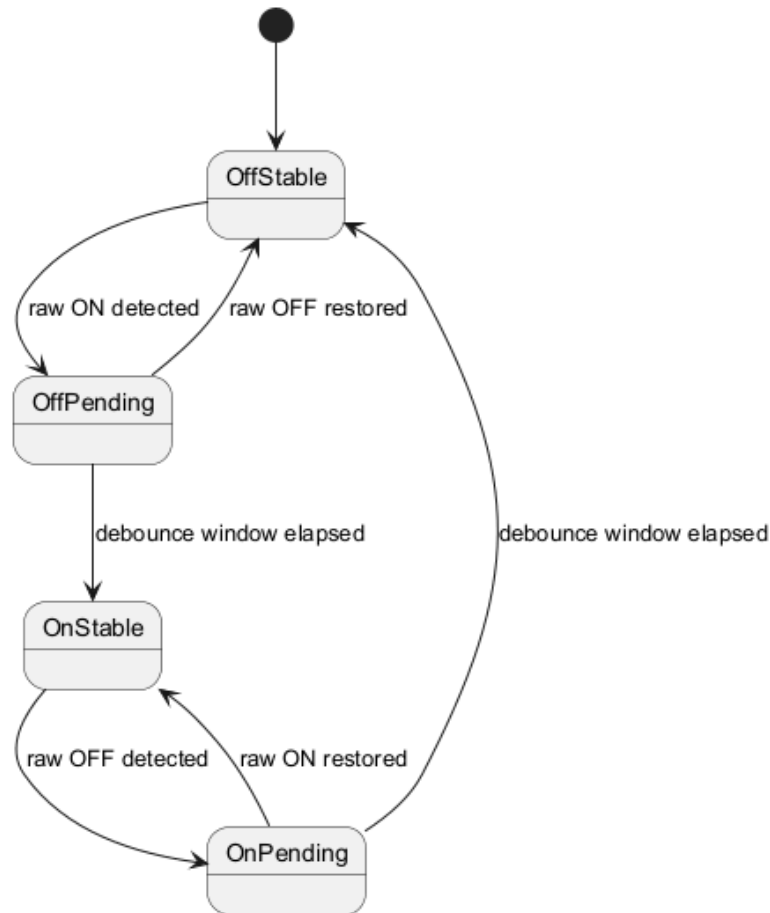


Figure 9: Binary actuator debounce state machine

3. Implementation

3.1 Source tree and responsibility split

Important source files are:

- `Code/src/app/app_controller.cpp` – initialization and cooperative scheduler;
- `Code/src/app/app_state.h` – compact shared state for both actuators;
- `Code/src/tasks/task_command.cpp` – serial commands and control actions;
- `Code/src/tasks/task_inputs.cpp` – pushbutton and potentiometer sampling;
- `Code/src/tasks/task_conditioning.cpp` – periodic execution of both conditioning services;
- `Code/src/tasks/task_actuator.cpp` – binary LED and analog PWM application;
- `Code/src/tasks/task_reporting.cpp` – LCD and serial reports;

- `Code/src/services/command_conditioning.cpp` – binary debounce logic;
- `Code/src/services/analog_conditioning.cpp` – analog filtering pipeline;
- `Code/src/services/runtime_telemetry.cpp` – history and statistics service;
- `Code/src/drivers/hardware_inputs.cpp` – button and potentiometer access.

3.2 3.2 Cooperative timing

The task periods are:

- command task: 20 ms;
- hardware input task: 20 ms;
- conditioning task: 20 ms;
- actuator task: 20 ms;
- reporting task: 1000 ms.

The binary actuator uses an 80 ms debounce window with a 20 ms update period, which preserves the sub-100 ms target in nominal conditions.

3.3 3.3 Main commands

The final command interface includes:

- `SET_RELAY ON|OFF, ON, OFF;`
- `SET_ANALOG <0..100>, ANALOG_UP, ANALOG_DOWN, ANALOG_STOP;`
- `INPUT SERIAL|HARDWARE|HYBRID;`
- `STATUS, REPORT;`
- `HISTORY, HISTORY CLEAR;`
- `STATS, STATS RESET;`
- `DEBOUNCE <20..500>, AUTOREPORT ON|OFF, PLOT ON|OFF, HELP.`

3.4 3.4 Telemetry implementation

The telemetry service stores the latest events in a fixed-size ring buffer. This avoids dynamic memory and is appropriate for Arduino Uno. Typical events are:

- binary command received;
- binary output changed;

- analog command received;
- analog output changed;
- input mode changed;
- invalid command received;
- history or statistics reset.

Latency statistics are updated when each actuator output is actually applied. The project prints last, minimum, maximum, and average latency values together with the number of samples.

3.5 3.5 Memory optimization for Arduino Uno

One important implementation step was reducing memory usage so the complete two-actuator solution still fits on Arduino Uno. The main optimizations were:

- moving constant message strings to flash using `PSTR` and `printf_P`;
- shrinking counters and history data types;
- keeping compact state structures;
- avoiding heavier runtime dependencies for the analog output.

The resulting build fits comfortably in RAM and flash.

4 4. Hardware setup and diagrams

4.1 4.1 Wokwi hardware setup

The final simulated hardware described in `Code/diagram.json` contains:

- Arduino Uno;
- LCD1602 I2C display on A4/A5;
- binary actuator LED on D13;
- analog actuator PWM LED on D10;
- pushbutton on D2 using `INPUT_PULLUP`;
- potentiometer signal on A0;
- serial monitor permanently visible for mixed input/output interaction.

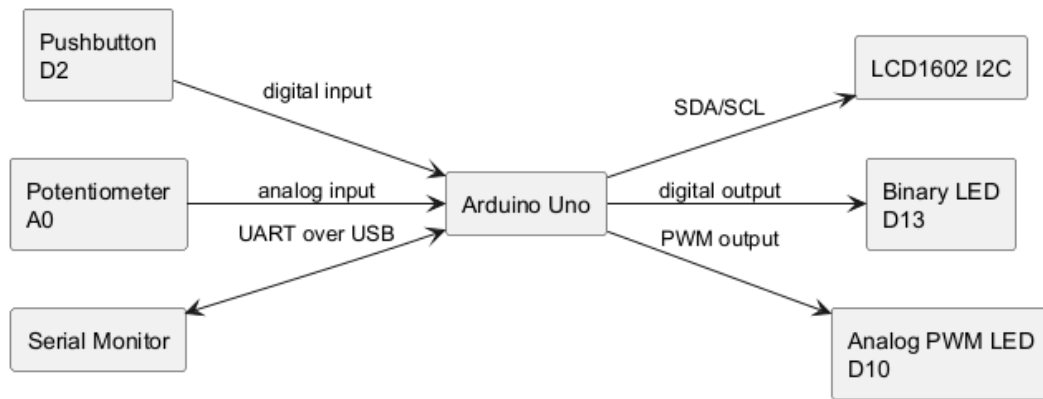


Figure 10: Electrical block view of the final Wokwi setup

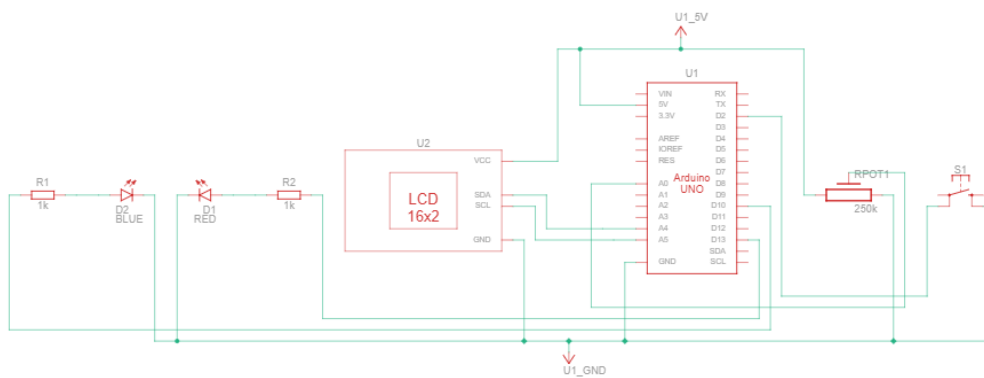


Figure 11: Electrical schematic exported from Tinkercad

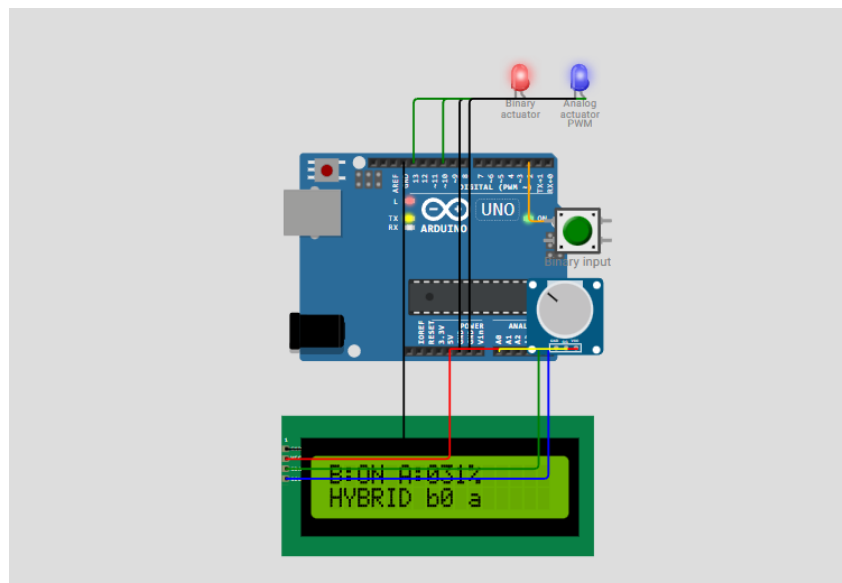


Figure 12: Final Wokwi runtime view with both actuators and both physical inputs

4.2 4.2 Diagram inventory

The final illustrated set covers architecture, flows, sequence behavior, validation, and hardware layout.

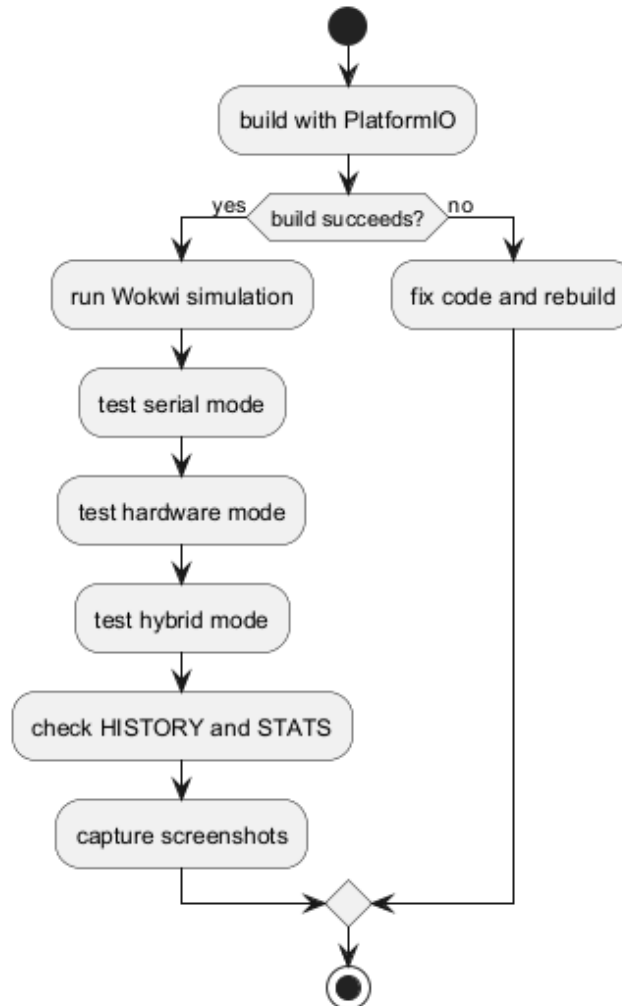


Figure 13: Validation workflow used for the final project

5 5. Validation and results

5.1 5.1 Validation scenarios

The following checks were used:

1. build the project with `pio run`;
2. run the Wokwi simulation;
3. test serial-only control;
4. test hardware-only control;

5. test hybrid mode;
6. inspect event history and latency statistics;
7. verify LCD and serial reporting;
8. check memory use on Arduino Uno.

5.2 5.2 Observed outcomes

ID	Check	Status	Evidence
T1	Serial command parsing	Pass	valid commands acknowledged
T2	Binary debounce path	Pass	stable switching without chatter
T3	Analog conditioning path	Pass	smooth PWM transitions
T4	Pushbutton hardware input	Pass	binary output follows hardware input mode
T5	Potentiometer hardware input	Pass	analog output follows mapped percentage
T6	Event history	Pass	recent events printed on demand
T7	Latency statistics	Pass	min/max/avg values printed on demand
T8	LCD and serial reporting	Pass	both interfaces updated correctly
T9	Two-actuator completeness	Pass	binary and analog actuators coexist
T10	Uno memory fit	Pass	RAM and flash remain within limits

Table 2: Validation summary

5.3 5.3 Build result

The final PlatformIO build for Arduino Uno reports approximately:

- RAM: about 53.6%;
- Flash: about 59.3%.

This confirms that the optimized final implementation is realistic for the selected target board.

```
REPORT reason-periodic mode=HYBRID binary_raw=ON binary_cond=ON binary_out=ON binary_alert=0 debounce_ms=80 binary_latency_ms=1530 analog_raw=31 ana
log_sat=31 analog_med=31 analog_avg=31 analog_ramp=31 analog_out=31 pwm=79 analog_alert=0 analog_latency_ms=7475 ok=1 err=0 tr=5 last-on
B:ON A:031%
HYBRID b0 a0
REPORT reason-periodic mode=HYBRID binary_raw=ON binary_cond=ON binary_out=ON binary_alert=0 debounce_ms=80 binary_latency_ms=1530 analog_raw=31 ana
log_sat=31 analog_med=31 analog_avg=31 analog_ramp=31 analog_out=31 pwm=79 analog_alert=0 analog_latency_ms=7475 ok=1 err=0 tr=5 last-on
B:ON A:031%
HYBRID b0 a0
```

Figure 14: Serial monitor output with reports, history, and statistics

```

C:\Sandu\III\Sem6\SI\si-course-repo\Lab7\Code>pio run
Processing uno (platform: atmelavr; board: uno; framework: arduino)
-----
Verbose mode can be enabled via '-v, --verbose' option
CONFIGURATION: https://docs.platformio.org/page/boards/atmelavr/uno.html
PLATFORM: Atmel AVR (5.1.0) > Arduino Uno
HARDWARE: ATMEGA328P 16MHz, 2KB RAM, 31.50KB Flash
DEBUG: current (avr-stub) External (avr-stub, simavr)
PACKAGES:
  - framework-arduino-avr @ 5.2.0
  - toolchain-atmelavr @ 1.70300.191015 (7.3.0)
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 6 compatible libraries
Scanning dependencies...
Dependency Graph
|-- LiquidCrystal_I2C @ 1.1.4
Building in release mode
Checking size .pio/build/uno/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [=====] 53.6% (used 1098 bytes from 2048 bytes)
Flash: [=====] 59.3% (used 19112 bytes from 32256 bytes)
===== [SUCCESS] Took 1.67 seconds =====

```

Figure 15: Successful PlatformIO build for the final optimized project

Conclusions

The final project is no longer only a binary-actuator demonstration. It is a complete embedded control application for Arduino Uno, containing a binary actuator path, an analog actuator path, serial and hardware input sources, explicit signal conditioning, LCD display, structured reporting, event history, and latency statistics.

The supplementary requirements are satisfied through modular implementation, clear signal-processing stages, validation-oriented reporting, and comprehensive documentation. The final code also fits the hardware limitations of Arduino Uno after memory-oriented optimization.

AI tool usage note

During the preparation of this report and the software implementation, AI tooling was used for drafting, restructuring, and consolidating both the codebase and the written material. The resulting code, diagrams, and report content were reviewed, validated, adjusted, and completed according to the laboratory requirements before inclusion in the final submission.

Appendix

Repository link

<https://github.com/Tirppy/si-course-repo/tree/main/Lab7/Code>

Main build and run commands

- Build: `pio run`
- Serial monitor: `pio device monitor -b 115200`
- Wokwi: use `Code/diagram.json` and `Code/wokwi.toml`